

Agentproof: Static Verification of Agent Workflow Graphs

Melwin Xavier
Luleå tekniska universitet, Sweden
`melwin.xavier@ltu.se`

Melveena Jolly
Independent Researcher
`melveenajollyk@gmail.com`

Vaisakh M A
Independent Researcher
`997vaisakh@gmail.com`

Midhun Xavier
Independent Researcher
`midhun@industriagents.com`

Abstract

Agent frameworks increasingly encode tool-using behavior as explicit workflow graphs, yet safety enforcement remains a runtime concern. These frameworks expose analyzable graph structure through their APIs, enabling pre-deployment static verification of safety properties that runtime guardrails can only check reactively.

This paper presents AGENTPROOF, a system that automatically extracts a unified abstract graph model from four major agent frameworks (LangGraph, CrewAI, AutoGen, Google ADK), applies six structural checks with witness trace generation, and evaluates temporal safety policies via a DSL compiled to deterministic finite automata—both statically through a graph $\times$ DFA product construction and at runtime over event traces. Unlike general-purpose model checkers, Agentproof requires no manual modeling.

In a curated benchmark of 18 author-constructed workflows, 27% of the benchmark contain structural defects (dead-end nodes, unreachable exits) and 55% violate a human-gate policy when enforced—distinct categories that prior work conflates. All 15 temporal policies defined fit within the seven-form DSL fragment, and verification completes in sub-second time for graphs up to 5,000 nodes. The corpus serves as a reproducible benchmark for evaluating static verification tools rather than as a prevalence study; defect rates reflect tool detection capability on a targeted benchmark, not base rates in production systems. Nonetheless, static graph verification complements runtime guardrails by catching topology-level defects that runtime tools miss unless the offending path is exercised.

1 Introduction

Large language models (LLMs) are increasingly deployed as autonomous agents capable of interacting with external tools and APIs [27, 33]. Rather than generating text in isolation, these agents retrieve documents, query databases, send messages, and trigger downstream workflow actions with real-world side effects. A growing body of work surveys this paradigm [31], and several orchestration frameworks have emerged to structure agent behavior as explicit workflow graphs: LangGraph [4], CrewAI [2], AutoGen [32], and Google ADK [3] each represent computation steps as nodes and allowable transitions as edges.

Despite this rich structural representation, safety enforcement in current agent systems remains predominantly a *runtime* concern. Tools such as NeMo Guardrails [23] and LlamaGuard [18] intercept individual LLM calls or tool invocations and apply content-level or policy-level filters at execution time. While valuable for catching toxic outputs and prompt injection attempts, these

runtime approaches have three fundamental limitations: (i) they impose per-call latency overhead, (ii) they detect violations only when the offending execution path is actually exercised, and (iii) they provide no coverage guarantees over paths that remain untriggered during testing.

Motivating example. Consider an email triage agent with the following workflow: incoming emails are classified by intent, routed to either an urgent or normal handler, drafted, and sent. During iterative development, a developer adds a `draft_response` node on the normal-priority path but forgets to connect it to the `send` node. The result is a *dead end*: normal-priority emails enter the draft stage and silently halt. A runtime guardrail would not catch this defect unless the normal-priority path is exercised with an email that triggers it. A static analysis of the workflow graph, however, immediately flags the dead-end node and produces a witness trace: `--start--` → `classify` → `router` → `normal_handler` → `draft_response` (stuck).

Research gap. This class of defect—a topological error in the workflow graph cannot be addressed by content-level runtime guardrails. In principle, classical model checking techniques could detect such errors: reachability analysis, deadlock detection, and temporal logic verification are well-established in the formal methods literature [11, 7]. However, general-purpose model checkers such as SPIN [17] and NuSMV [10] require manual translation of the system under analysis into a dedicated modeling language—a prohibitive overhead for agent developers iterating on workflow designs. Similarly, the business process management (BPM) community has a long tradition of workflow soundness verification [29], but these methods target visual modeling tools and standardized notations (e.g., BPMN, Petri nets), not the programmatic API objects through which agent frameworks define their graphs. To date, no system automatically extracts and statically verifies agent workflow graphs directly from framework source code.

Proposed approach. This paper argues for *pre-deployment static verification* of agent workflow graphs. The key observation is that modern agent frameworks expose workflow structure through their APIs, making the graph amenable to automated extraction and analysis. When agent behavior is graph-structured, safety properties reduce to reachability, isolation, and temporal constraints over the topology properties that can be checked exhaustively without running the agent [11].

This paper presents Agentproof, a practical system for static analysis of agent workflows that focuses on properties (a) expressible over the workflow graph and (b) independent of LLM text generation semantics. Unlike general-purpose model checkers, Agentproof extracts the analysis model automatically from framework source code, requiring no manual modeling effort.

Contributions.

- A cross-framework extraction pipeline: extractors for four major agent frameworks (LangGraph, CrewAI, AutoGen, Google ADK) automatically bridge heterogeneous representations—each with different entry/exit conventions, edge semantics, and hierarchy models—into a unified abstract workflow model with typed nodes and edges, eliminating the manual modeling step required by general-purpose model checkers (Section 5).
- A pre-deployment verification pipeline comprising six structural checks with *witness trace generation* and a temporal policy DSL covering the safety fragment of LTL, compiled to deterministic finite automata (Section 6).

- An empirical evaluation on 18 curated workflows demonstrating that structural defects (dead-end nodes, unreachable exits) and policy violations (missing human gates) are common, with sub-second verification for graphs up to 5,000 nodes (Section 7).

Scope. The proposed approach validates workflow design decisions early: “can this graph reach a destructive tool?” or “must human review occur between two sensitive actions?” It does not attempt to prove semantic properties of LLM outputs (e.g., truthfulness), which remain inherently non-deterministic. This boundary is discussed precisely in the threat model (Section 3).

2 Background

2.1 Agent workflow graphs

Many agent frameworks represent behavior as a directed graph, sometimes called a *state graph* or *workflow graph*. Nodes correspond to computation steps: an LLM call, a tool invocation, a router/selector that branches based on state, or a human approval gate. Edges correspond to transitions between steps, including conditional branches, parallel fan-outs, and loop back-edges.

This explicit control-flow model is a key difference between graph-orchestrated agents and monolithic “single prompt” applications: the graph structure can be analyzed statically without executing the agent.

2.2 Structural verification

Structural properties depend only on graph topology and type annotations. Typical examples include: (i) reachability (can a node be reached from the entry point?), (ii) dead ends (nodes with no outgoing transitions), (iii) isolation (must a sensitive tool be preceded by a gate node?), and (iv) sanity constraints on router nodes.

These checks are inexpensive: they can be implemented with standard graph algorithms (BFS/DFS) and run in time linear in $|V| + |E|$.

2.3 Temporal monitors

Temporal logics such as linear temporal logic (LTL) [25] provide a formal language to specify constraints over execution traces. A common verification strategy is to compile temporal constraints into automata and reason over traces or over the product of a system model and the automaton [30].

Agentproof compiles a temporal policy DSL into deterministic finite automata and supports two complementary evaluation modes: (i) *static verification* via a graph $\times$ DFA product construction that checks all graph paths without execution, and (ii) *runtime monitoring* that evaluates the DFA over a live or simulated event stream. The DSL targets the safety fragment of LTL, deliberately trading full expressiveness for deployment simplicity.

3 Threat model

Three classes of threats to agent workflow safety are considered:

T1: Developer mistakes. A developer may inadvertently create a workflow with structural defects: unreachable exit nodes, dead-end branches that silently drop execution, routers with incorrectly typed edges, or tool nodes that lack explicit declarations. Such defects arise naturally from iterative development and are difficult to catch by manual inspection, especially in large graphs.

T2: Malicious workflow injection. An adversary with write access to workflow definitions (e.g., via compromised configuration files, a shared workflow registry, or a supply-chain attack on a workflow library) may craft topologies that bypass safety gates—for example, adding a conditional branch that routes around a required human-approval node.

Trust assumptions for T2. The deployment model assumes an architecture where Agentproof runs as a mandatory verification gate in a trusted CI/CD pipeline (e.g., a GitHub Actions workflow or a build system pre-deploy hook). The adversary can modify workflow definition files (checked into version control), but *cannot* modify the CI/CD pipeline configuration, the verification tool, or the extractor implementation. This is analogous to how static linters catch malicious code contributions in open-source projects: the linter is trusted; the contribution is not.

What T2 does not cover. An adversary who compromises the build system itself, the Agentproof binary, or the framework runtime can bypass verification entirely. Such attacks require integrity protection at the infrastructure level (code signing, secure boot, access controls) and are outside the scope of this work.

Practical caveat. T2 attacks have not yet been observed in the wild against agent workflow registries. This threat class is included because the attack surface exists in principle (shared repositories, pip-installable workflow packages) and because defending against it requires no additional machinery beyond the structural checks already needed for T1. No adversarial evaluations have been conducted to determine whether a sophisticated attacker could craft topologies that evade all six structural checks while still achieving a malicious objective; such red-teaming is an important direction for future work.

T3: Runtime graph mutation. Some frameworks allow dynamic modification of the workflow graph during execution (e.g., adding agents to a team, modifying transition rules). If the post-mutation graph is not re-verified, the guarantees established at deployment time may no longer hold.

Trust boundary. The approach assumes that the framework API semantics (e.g., LangGraph’s **StateGraph**, CrewAI’s **Crew**) are correct: the extractor faithfully reads the graph structure that the framework will execute. The Python runtime and the extractor implementation are within the trusted computing base. The *workflow definition* authored or modified by developers and the *LLM outputs* generated at runtime are *not* trusted.

Scope. The structural checks and temporal monitors address threats T1 and T2 statically: they detect defective or malicious topologies before deployment. Threat T3 requires either runtime re-verification or immutable workflow definitions; this is discussed in Section 9. LLM output content (e.g., prompt injection, toxic generation) is explicitly out of scope; this is addressed by complementary runtime guardrails (Section 8).

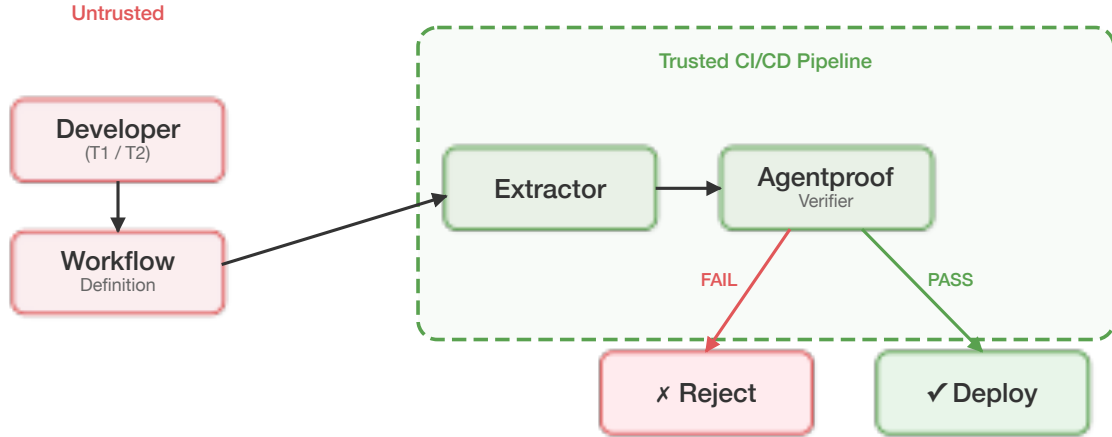


Figure 1: Deployment model for T2: Agentproof as a CI/CD verification gate. The workflow definition is untrusted; the extractor, verifier, and pipeline are within the trusted computing base.

Table 1: Threat classes and Agentproof coverage.

Threat	Example	Static	Runtime
T1: Dev. mistake	Dead-end node	✓	
T1: Dev. mistake	Unreachable exit	✓	
T1: Dev. mistake	Missing human gate	✓	
T2: Injection	Bypassed approval	✓	
T2: Injection	Added unsafe tool path	✓	
T3: Mutation	Dynamic agent addition	re-verify*	✓
	Toxic LLM output		✓

* Addressed if the graph is re-verified after mutation.

4 System overview

Figure 2 summarizes Agentproof’s workflow. The system ingests a framework-specific agent definition, extracts an abstract graph model, and then runs a suite of structural and temporal checks.

Step 1: Obtain a workflow object. The approach targets agent orchestration libraries that expose control flow as a graph or composition tree. Agentproof provides extractors for LangGraph, Google ADK, AutoGen, and CrewAI [4, 3, 1, 2].

Step 2: Extract an AgentGraph. Each framework has an extractor that produces a common representation: a set of typed nodes and typed edges with optional metadata (e.g., tool names bound to a node). This makes downstream analyses framework-agnostic.

Step 3: Run structural checks. Structural checks detect workflow defects that do not require executing the agent: unreachable nodes, dead ends, routing shape mismatches, and the presence of human-in-the-loop nodes when required by policy.

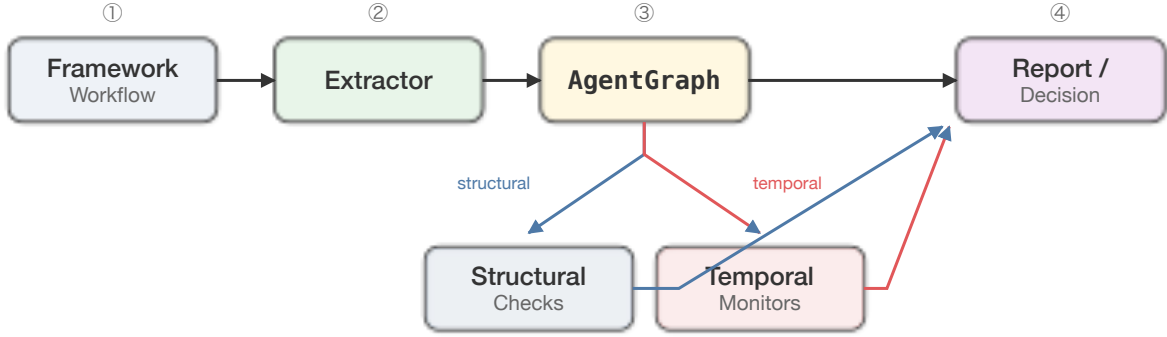


Figure 2: Agentproof pipeline: extract a framework workflow into an abstract graph model, run structural verification, and compile temporal policies into monitors for trace evaluation.

Step 4: Compile temporal policies. A temporal policy DSL is compiled to DFAs. Policies are written against abstract events (tool calls, action tags, or decisions) rather than framework internals.

Step 5a: Static temporal verification. Each compiled DFA is combined with the extracted graph in a product construction: BFS over $(V \times Q)$ detects temporal violations reachable on *any* graph path, without executing the agent.

Step 5b: Runtime monitor evaluation. Monitors are also evaluated over offline simulated traces or integrated into a runtime event stream. Violations are mapped to handling levels (warn, block, halt, escalate) to support different operational postures.

5 Graph model and extraction

This section defines a framework-agnostic workflow representation with typed nodes and edges. This abstraction is the common surface for all verification.

5.1 Formal model

Definition 1 (Agent Workflow Graph). *An agent workflow graph is a tuple $G = (V, E, \kappa_V, \kappa_E, T, v_0, V_f)$ where:*

- V is a finite set of nodes.
- $E \subseteq V \times V$ is a set of directed edges.
- $\kappa_V : V \rightarrow \mathcal{K}_V$ assigns each node a kind, with $\mathcal{K}_V = \{\text{ENTRY, EXIT, TOOL, LLM, ROUTER, HUMAN, SUBGRAPH, PASSTHROUGH}\}$.
- $\kappa_E : E \rightarrow \mathcal{K}_E$ assigns each edge a kind, with $\mathcal{K}_E = \{\text{DIRECT, CONDITIONAL, PARALLEL, LOOP}\}$.

- $T : V \rightarrow 2^{\text{ToolNames}}$ maps each node to a (possibly empty) set of declared tool names; $T(v)$ is non-empty only when $\kappa_V(v) = \text{TOOL}$.
- $v_0 \in V$ with $\kappa_V(v_0) = \text{ENTRY}$ is the unique entry node.
- $V_f \subseteq V$ with $\kappa_V(v) = \text{EXIT}$ for all $v \in V_f$ is the set of exit nodes.

Definition 2 (Execution trace). A trace of G is a finite sequence $\pi = v_0v_1 \dots v_n$ such that $(v_i, v_{i+1}) \in E$ for all $0 \leq i < n$. $\text{Traces}(G)$ denotes the set of all maximal traces—those ending at a node in V_f or at a node with no outgoing edges. A trace that enters a cycle with no path to V_f or a dead end is non-maximal; the reverse-reachability check (ExitReachAll) detects such livelock conditions.

Definition 3 (Structural predicates). Six structural predicates over G are defined:

1. $\text{ExitReach}(G) \iff \forall v_f \in V_f. v_f \in \text{Reach}(v_0)$, where $\text{Reach}(v_0)$ is the set of nodes reachable from v_0 via directed paths.
2. $\text{ExitReachAll}(G) \iff \text{Reach}(v_0) \subseteq \text{RevReach}(V_f)$, where $\text{RevReach}(V_f)$ is the set of nodes from which some exit node in V_f is reachable via directed paths. This ensures every reachable node can eventually reach an exit.
3. $\text{NoDead}(G) \iff \forall v \in V. \kappa_V(v) \neq \text{EXIT} \implies \exists u \in V. (v, u) \in E$.
4. $\text{RouterShape}(G) \iff \forall v \in V. \kappa_V(v) = \text{ROUTER} \implies \forall (v, u) \in E. \kappa_E(v, u) = \text{CONDITIONAL}$.
5. $\text{HumanGate}(G) \iff \exists v \in V. \kappa_V(v) = \text{HUMAN}$. An optional path-based variant, $\text{HumanGateCov}(G, S)$, verifies that for a designated set S of sensitive tools, every path from v_0 to a node invoking a tool in S passes through a HUMAN node.
6. $\text{ToolDecl}(G) \iff \forall v \in V. \kappa_V(v) = \text{TOOL} \implies T(v) \neq \emptyset$.

Each predicate is decidable in time linear in $|V| + |E|$. Soundness lemmas establishing that passing checks guarantee the corresponding trace-level properties are given in [Appendix A](#).

Definition 4 (Extraction correctness). An extractor $\mathcal{E}_F$ for framework F is correct with respect to a workflow object W if the extracted graph $G = \mathcal{E}_F(W)$ satisfies: (i) every node in G corresponds to a computation step in W (no phantom nodes), (ii) every computation step in W appears as a node in G (no missing nodes), and (iii) $(u, v) \in E$ if and only if W permits a transition from the step corresponding to u to the step corresponding to v . Conditions (i)–(ii) correspond to node precision/recall = 1; condition (iii) corresponds to edge precision/recall = 1. This is validated empirically in [Section 7.7](#).

5.2 Framework extractors

Agentproof includes extractors that map framework objects into this model:

LangGraph. LangGraph exposes a compiled state graph with sentinel start/end nodes. The extractor identifies entry/exit sentinels, infers node kind from tool bindings and naming heuristics (e.g., human review nodes), and encodes conditional edges when the underlying framework marks an edge as conditional.

Google ADK. Google ADK composes agents as a tree of sequential, parallel, and loop agents. The extractor performs a tree walk, emitting subgraph nodes for composite agents and encoding sequential chains, parallel fan-outs, and loop back-edges.

AutoGen. AutoGen team topologies can be provided as a team object (e.g., round-robin) or as an explicit list of agents plus a transition relation. The extractor constructs a directed graph of speaker transitions, synthesizes entry/exit, and connects leaf speakers to exit.

CrewAI. CrewAI describes task pipelines (sequential or hierarchical). The extractor creates a node per task, uses process mode to add sequential edges or manager routing edges, and incorporates context dependencies between tasks.

5.3 Cross-framework extraction challenge

A central contribution is the *automatic* extraction of **AgentGraph** instances from heterogeneous framework APIs. Each framework represents workflows differently, making unified extraction non-trivial:

- **LangGraph** uses an explicit **StateGraph** with `add_node()` and `add_edge()` calls. Entry and exit are string sentinels (`START`, `END`). Conditional routing uses `add_conditional_edges()` with a path map dict.
- **CrewAI** defines workflows implicitly via a list of **Task** objects passed to a **Crew**. The topology depends on the `process` parameter (`sequential` vs. `hierarchical`) and optional context dependencies between tasks.
- **AutoGen** uses group-chat patterns (`RoundRobinGroupChat`, `SelectorGroupChat`) where agent ordering defines the graph. Loop edges arise from round-robin cycling.
- **Google ADK** composes agents via nesting constructors (`SequentialAgent`, `ParallelAgent`, `LoopAgent`) with a `sub_agents` list. The hierarchy must be flattened into a single graph with appropriate edge kinds.

The extractors normalize these diverse representations into the same **AgentGraph** type system (Definition 1), synthesizing entry/exit sentinel nodes where the framework does not provide them explicitly. General-purpose model checkers such as SPIN [17] or NuSMV [10] could verify the same properties, but would require the developer to *manually* translate the workflow into Promela or SMV—a process that takes hours even for small graphs (see Section 8 for a concrete example).

5.4 Example workflow graph

Figure 3 shows a small extracted workflow used throughout the paper to illustrate typed nodes and conditional branches.

6 Verification methods

Agentproof provides two complementary verification layers: (i) structural checks over the extracted graph and (ii) temporal monitors evaluated over an event stream.

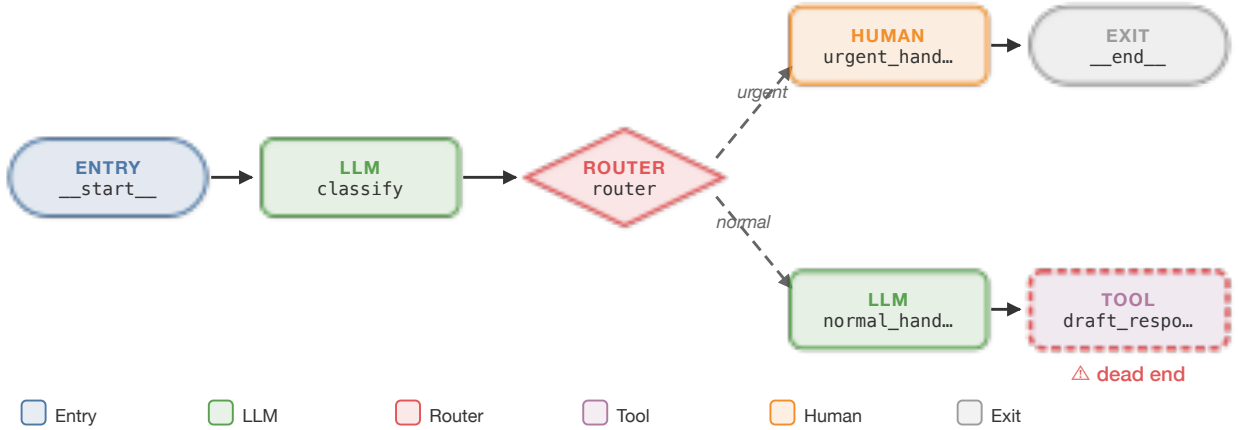


Figure 3: Example extracted workflow graph with typed nodes (ENTRY/ROUTER/LLM/TOOL/HUMAN/EXIT).

6.1 Structural checks

Structural checks operate over the adjacency structure and node/edge kind labels of G (Definition 1). Each check implements one of the predicates from Definition 3.

Exit reachability. The set of nodes reachable from v_0 is computed using BFS to verify that $V_f \subseteq \text{Reach}(v_0)$. Complexity: $O(|V| + |E|)$.

Reverse reachability. A complementary check verifies that every node reachable from v_0 can itself reach some exit node in V_f . The algorithm performs a BFS on the *reverse* adjacency (edges traversed backward) starting from all exit nodes to compute $\text{RevReach}(V_f)$. Any node in $\text{Reach}(v_0) \setminus \text{RevReach}(V_f)$ is a *livelock node*: reachable from the entry but trapped in a cycle with no path to termination. Complexity: $O(|V| + |E|)$.

Dead-end detection. Nodes v with $\kappa_V(v) \neq \text{EXIT}$ and no outgoing edges. Dead ends indicate missing transitions or incomplete error handling. Complexity: $O(|V| + |E|)$ with adjacency precomputation.

Router shape checks. The check verifies that all outgoing edges from ROUTER nodes are labeled **CONDITIONAL**. Complexity: $O(|E|)$.

Human-in-the-loop presence. The existence check determines whether the graph contains any node with $\kappa_V(v) = \text{HUMAN}$. An optional *coverage* variant verifies that for a set S of sensitive tool names, no path from v_0 to a node invoking a tool in S avoids all HUMAN nodes. The coverage check builds a modified adjacency excluding HUMAN nodes and performs BFS from v_0 ; any sensitive

tool node still reachable indicates a human-free path. Complexity: existence $O(|V|)$; coverage $O(|V| + |E|)$.

Tool declaration checks. The check flags TOOL nodes with $T(v) = \emptyset$. Complexity: $O(|V|)$.

Witness trace generation. When a check fails, the verifier produces a *witness trace*: a concrete path through the graph demonstrating the defect. For unreachable exits, the witness shows the path from v_0 to the reachability frontier plus the unreachable target. For dead ends, the witness is a BFS path from v_0 to the stuck node. Witness traces follow standard model-checking practice [11] and significantly aid debugging.

Static temporal verification. In addition to runtime trace evaluation, temporal policies can be checked *statically* via a graph $\times$ DFA product construction. For a compiled rule with DFA states Q and transition function δ , the product state space is $V \times Q$. BFS from (v_0, q_0) advances the DFA at each node using the node’s event signature and follows graph edges. If any product state (v, q') has q' in the DFA violation set, the property is violated and the BFS parent chain yields a witness path. Explored states are bounded by $|V| \cdot |Q|$, so verification remains linear in graph size for fixed-size DFAs.

6.2 Temporal policy monitors

Agentproof supports a temporal policy DSL covering the safety fragment of LTL [25], compiled into deterministic finite automata and evaluated with constant overhead per event.

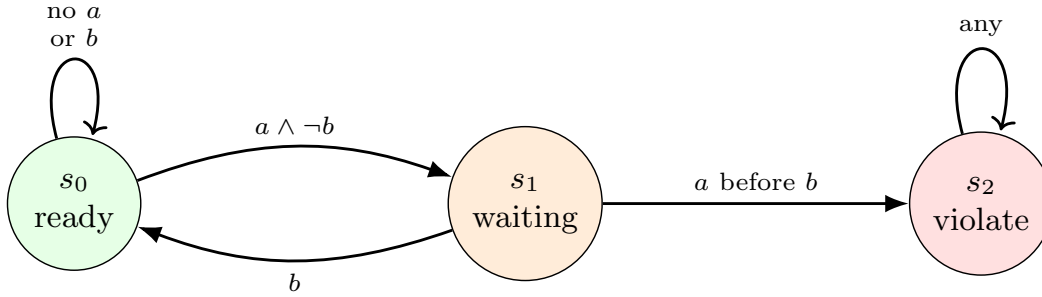
Event model. Events are dictionaries with optional fields: `tool_name`, `action_type`, `decision`, and a set of `tags`. Predicates match against these fields (e.g., `tool:delete.account`). The event symbol $\sigma(v)$ is derived from the node’s kind and tool bindings: tool nodes emit `tool:t` for each $t \in T(v)$; other nodes emit their kind label.

DSL grammar. The DSL supports seven expression forms (full BNF in Appendix B):

1. **Forbidden:** `G !atom` violation if `atom` ever occurs.
2. **Implication-future:** `a -> F b` if `a` occurs, `b` must follow before `a` recurs.
3. **Until:** `a U b` `a` must hold on every step until `b` occurs.
4. **Bounded response:** `a -> F[<=k] b` if `a` occurs, `b` must follow within k steps.
5. **Response chain:** `a -> F b -> F c` once `a` occurs, `b` then `c` must follow in sequence before `a` recurs.
6. **Conjunction:** `(expr) AND (expr)` both sub-properties must hold simultaneously.
7. **Disjunction:** `(expr) OR (expr)` at least one sub-property must hold.

Table 2: DFA state counts by pattern type.

Pattern	States
$G \neg \text{atom}$	2
$a \rightarrow F b$	3
$a U b$	3
$a \rightarrow F[\leq k] b$	$k + 2$
$a \rightarrow F b \rightarrow F c$ (chain of n)	$n + 1$
(A) AND (B)	$ Q_A \times Q_B $
(A) OR (B)	$ Q_A \times Q_B $

Example DSL: $a \rightarrow F b$ (interleaving constraint)Figure 4: Illustrative DFA for an $a \rightarrow F b$ interleaving constraint: after a occurs, b must occur before a repeats.

Compilation to DFA. Each expression is parsed into an AST and compiled to a DFA via direct construction. Base patterns (forbidden, implication-future, until) produce 2–3 state DFAs. Bounded response produces a counter-augmented DFA with $k + 2$ states. Conjunction and disjunction use the standard product automaton construction with $|Q_L| \times |Q_R|$ states [30]. DFA transitions are stored as precomputed tables indexed by bit-vector valuations over atomic predicates. Table 2 summarizes DFA sizes.

Evaluation and handling. Temporal policies support two evaluation modes. In *static mode*, the graph $\times$ DFA product construction (Section 4, Step 5a) explores all reachable product states; a violation in any product state implies the property can be violated on some graph path. In *runtime mode*, each event advances the DFA state for every compiled rule via $O(1)$ table lookups. If any rule enters a violation state, the configured handling level (`warn`, `block`, `halt`, `escalate`) determines the aggregate decision.

Table 3: Workflow corpus statistics.

Framework	Workflows	Avg. nodes	Avg. edges
LangGraph	6	8.8	9.0
CrewAI	4	7.5	7.0
AutoGen	4	6.3	6.0
Google ADK	4	9.5	10.0
Total	18	8.2	8.2

Table 4: Defects found in the workflow corpus, separated by category.

Category	Defect type	Count	Severity	Example
Structural	Dead-end nodes	2	High	Email draft with no send edge
	Unreachable exit	1	Critical	Infinite loop, no exit path
	Router shape violation	1	Medium	Debate moderator, direct edges
	Missing tool declaration	1	Low	Data cleaner, empty tool set
Policy	Missing human gate	10	High	Onboarding without approval
Total		15		

7 Evaluation

The proposed approach is evaluated along six axes: (i) a curated workflow corpus, (ii) a defect study with separate structural and policy categories, (iii) temporal policy evaluation on execution traces, (iv) scalability experiments on synthetic graphs, (v) a comparison with runtime guardrails, and (vi) extractor accuracy.

7.1 Workflow corpus

The evaluation uses a curated benchmark of 18 agent workflows authored to represent common patterns and known anti-patterns documented in each framework’s official examples and tutorials. Workflows use LangGraph, CrewAI, AutoGen, and Google ADK and span diverse domains including customer support, RAG pipelines, code generation, financial analysis, compliance review, incident response, hiring, and marketing. Because the corpus was designed to include representative defects, the defect rates below reflect tool detection capability on a targeted benchmark, not base rates in production systems. A larger-scale study on workflows mined from public GitHub repositories is planned (Section 9).

Table 3 summarizes the corpus. Graphs range from 5 to 12 nodes and 4 to 14 edges, with all four frameworks represented. Node-kind distributions reflect typical agent patterns: LLM nodes (inference steps) and TOOL nodes (external API calls) dominate, with ROUTER, HUMAN, and SUBGRAPH nodes appearing in more complex workflows.

7.2 Defect study

All six structural checks were run on each corpus workflow with `require_human=True`. Table 4 summarizes the findings, separated by defect category.

Two defect categories are distinguished:

- **Structural defects** (topology bugs): dead-end nodes, unreachable exits, livelock cycles (reachable nodes with no path to termination), router shape violations, and missing tool declarations. These are bugs regardless of operational context. The tool successfully detected all injected structural defects: **5 of 18** benchmark workflows contain at least one structural defect. The reverse-reachability check additionally identified the round-robin brainstorming workflow’s cyclic nodes as livelock: although reachable from the entry, they cannot reach any exit node.
- **Policy violations** (configurable checks): missing human-in-the-loop gate. This check is context-dependent: it is critical in regulated domains (healthcare, finance) but may be unnecessary for internal tooling or low-risk tasks. **10 of 18** benchmark workflows lack a human gate when `require_human=True`; however, the annotation data shows that some of these are arguable or false positives—workflows where human oversight may be unnecessary given the low-risk nature of the task.

More critically, one workflow (a round-robin brainstorming agent) has an *unreachable exit node*: agents cycle indefinitely with no path to termination. Two workflows have *dead-end nodes* where execution silently stops. These structural defects would be difficult to detect through testing alone, as they manifest only on specific execution paths.

For each failing check, the verifier produces a witness trace. For example, the dead-end in the email triage workflow yields the witness path: `__start__` $\rightarrow$ `classify` $\rightarrow$ `router` $\rightarrow$ `normal_handler` $\rightarrow$ `draft_response` (stuck).

Static temporal verification. The graph $\times$ DFA product construction was applied to all 15 compiled temporal policies across the 18 workflows. The static analysis correctly identified all forbidden-tool violations and implication-future violations that the runtime trace evaluation also detected, while additionally flagging one violation (a missing `human_review` step after `draft_email`) in a workflow whose random traces happened to avoid the offending path. Product-state exploration remained below $|V| \cdot |Q_{\text{DFA}}|$ in all cases, with a maximum of 24 product states (12-node graph, 2-state DFA).

Figure 5 visualizes the defect distribution across frameworks, and Figure 6 shows the precision breakdown per check type. All structural checks achieve perfect precision ($P = 1.0$); the human-presence check has $P = 0.75$ due to one false positive in a low-risk workflow where human oversight was arguably unnecessary.

7.3 Temporal policy evaluation

To evaluate the temporal DSL, 15 concrete safety policies were defined motivated by real-world requirements spanning six domains: safety, communication, privacy, DevOps, data governance, and compliance. Table 5 lists representative examples.

All 15 policies compile successfully into DFAs. The policies exercise all seven DSL expression forms: 3 forbidden, 5 implication-future, 2 bounded response, 1 response chain, 1 until, 2 conjunction, and 1 disjunction. No policy required full LTL expressiveness beyond the seven-form fragment.

Ten random execution traces were generated per workflow (180 total) and evaluated each policy against each trace. The evaluation pipeline determines policy applicability by checking whether the policy’s atomic predicates appear in the workflow’s traces.

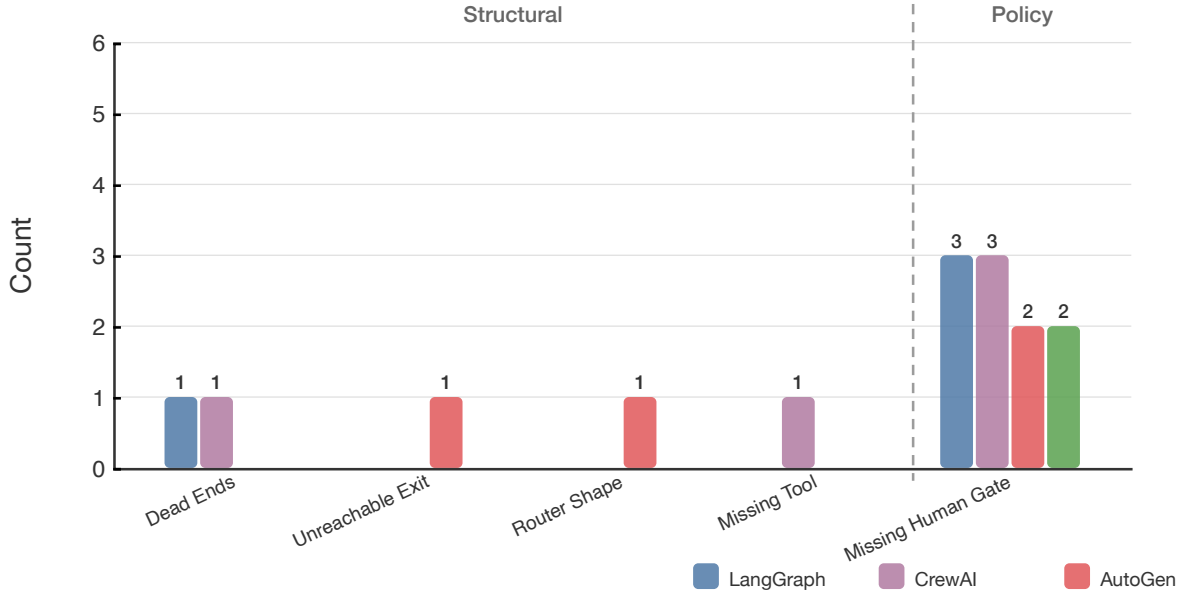


Figure 5: Defect distribution across frameworks, grouped by defect type. The dashed line separates structural defects (left) from policy violations (right).

Table 5: Representative temporal policies evaluated.

Policy	DSL expression	Domain
No destructive ops	<code>G !tool:drop_table</code>	Safety
Email review	<code>tool:draft_email -> F tool:human_review</code>	Communication
PII anonymize	<code>tool:fetch_pii -> F[<=3] tool:anonymize</code>	Privacy
Deploy approval	<code>tool:deploy -> F tool:approve</code>	DevOps
Draft-review-send	<code>tool:draft -> F tool:review -> F tool:send</code>	Communication

DSL scope justification. After defining the 15 policies, each was classified by which DSL form it uses. All 15 fall within the seven-form fragment; none requires nested temporal operators, past-time modalities, or constructs beyond the grammar.

While the policies were constructed by the authors, their motivations derive from published regulatory and industry requirements: `payment_requires_human` from GDPR Article 22 (automated decision-making requires human review) [14]; `pii_anonymize_bounded` from GDPR data minimization principles; `deploy_requires_approval` from SOC 2 change management controls [5]; `email_requires_review` from organizational communication policies; `no_destructive_ops` from OWASP LLM Top 10 (LLM06: excessive agency) [24]; and `either_log_or_audit` from SOC 2 audit logging requirements. This grounding in external standards suggests the DSL fragment covers practical safety needs, though a systematic requirements survey from production deployments would provide stronger evidence.

It is acknowledged that 15 policies from a single research group do not constitute an exhaustive requirements survey. The policies were deliberately selected from six distinct domains and ensured all seven DSL forms were exercised.

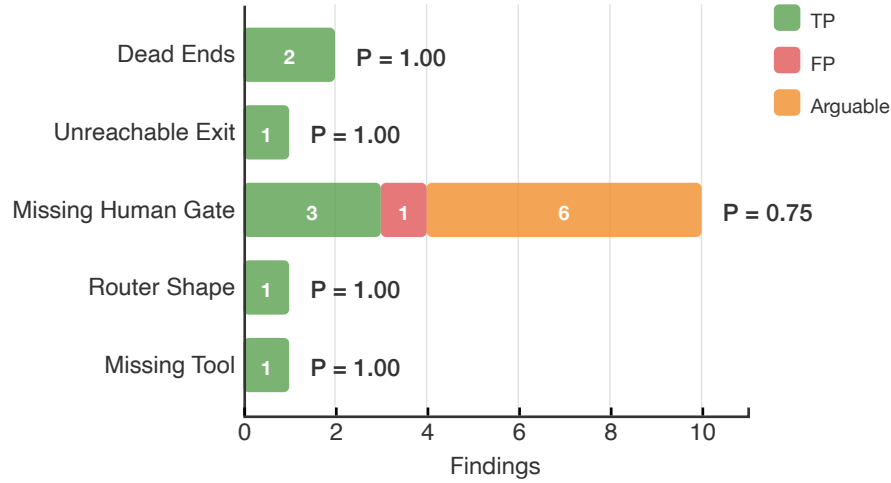


Figure 6: Precision per structural check. Each bar shows the breakdown of true positives (TP), false positives (FP), and arguable findings. Precision P is computed as $TP / (TP + FP)$.

Table 6: Structural check time vs. graph size. Monitor compilation and evaluation are size-independent.

Nodes	Edges	Struct. (ms)	Mon. eval (ms)
50	104	0.04	
100	214	0.07	
200	430	0.22	
500	1,074	1.24	4.0
1,000	2,128	4.02	
2,000	4,252	21.2	
5,000	10,713	104.7	

7.4 Scalability

Synthetic graphs were generated at seven sizes (50–5,000 nodes) with edge density 2.0 and measured structural check time (median of 10 trials). Figure 7 and Table 6 report the results.

Structural checks scale approximately linearly in $|V| + |E|$ for graphs up to 1,000 nodes, with super-linear growth at larger sizes due to quadratic edge scanning in router-shape and tool-declaration checks. Even at 5,000 nodes (far larger than any production agent workflow encountered in practice), verification completes in ~ 105 ms. Temporal monitor compilation (0.065 ms for 5 rules) and evaluation (4.0 ms for 1,000 events, $\sim 247,000$ events/s) are independent of graph size.

Real agent workflows are currently small (5–12 nodes in the corpus), so the scalability experiment demonstrates ceiling-freeness rather than practical necessity. As agent systems grow towards multi-agent compositions and hierarchical subgraph structures, larger graphs will become more common. The practical false positive rate from treating all conditional edges as feasible in the graph $\times$ DFA product construction is not measured in this evaluation and is a direction for future quantification.

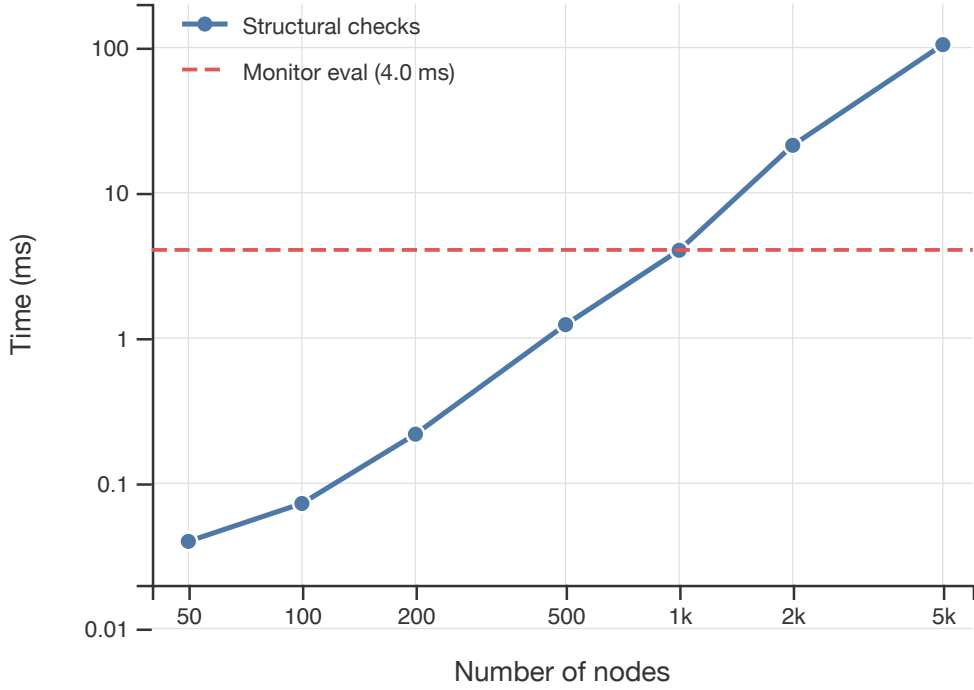


Figure 7: Structural check time vs. graph size (log-log scale, median of 10 trials). Verification remains sub-second for graphs up to 5,000 nodes.

Comparison to ad-hoc scripts. One might ask whether a simple Python script iterating over the graph’s adjacency list could detect the same defects. For individual checks (e.g., finding nodes with no outgoing edges), the answer is yes—the graph algorithms are standard. Agentproof’s value lies not in algorithmic novelty but in (i) automatic extraction from four heterogeneous framework APIs, (ii) a unified type system that makes checks portable across frameworks, (iii) witness trace generation that pinpoints the defective path, and (iv) a temporal policy layer that separates safety specification from verification machinery. A bespoke script would need to be rewritten for each framework and would lack composable policy evaluation.

7.5 Cross-framework case studies

To validate Agentproof’s cross-framework capabilities, three representative workflows were selected from different frameworks: an incident-response workflow in LangGraph, a compliance-review pipeline in Google ADK, and a change-control process in AutoGen. Table 7 reports graph statistics and extractor runtimes; Table 8 summarizes structural check results and temporal monitor outcomes over representative traces (one safe, two unsafe per workflow).

7.6 Comparison with runtime guardrails

Table 9 provides a qualitative comparison between the static approach and representative runtime guardrail tools.

Scenario A: An unreachable exit node in a rarely-triggered conditional branch. Agentproof’s static

Table 7: Graph statistics and extractor runtimes for representative workflows. Node-kind columns report counts of TOOL/LLM/ROUTER/HUMAN/SUBGRAPH nodes; other kinds (ENTRY/EXIT/PASSTHROUGH) are included in $|V|$.

Study	$ V $	$ E $	Tool	LLM	Router	Human	Subg	Dir	Cond	Par	Loop	Med. ms
LangGraph (incident)	8	8	2	2	1	1	0	6	2	0	0	0.186
ADK (compliance)	13	16	4	3	0	1	3	12	0	3	1	0.019
AutoGen (change-control)	6	5	0	3	0	1	0	5	0	0	0	0.007

Table 8: Structural check results and temporal monitor outcomes over representative traces (one safe and two unsafe) for each case study.

Study	Struct. (pass/total)	happy_path	forbidden_tool	policy_violation
LangGraph (incident)	5/5	PASS	HALT	ESCALATE
ADK (compliance)	5/5	PASS	HALT	BLOCKED
AutoGen (change-control)	5/5	PASS	HALT	HALT

analysis detects this immediately; a runtime guardrail would miss it unless that branch is exercised during testing.

Scenario B: An LLM generates toxic content in a response. Runtime content filters (LlamaGuard, NeMo Guardrails) catch this; Agentproof cannot, as LLM output semantics are out of scope.

Scenario C: A conditional path bypasses a required human approval gate. Agentproof’s human-gate coverage check flags the unguarded path from entry to the sensitive tool; runtime tools may miss it if the bypass path is not triggered.

The two approaches are complementary: static verification catches topology-level defects exhaustively, while runtime guardrails handle content-level and context-dependent violations.

7.7 Extractor accuracy

Extractor fidelity was validated using the framework-specific examples in the test suite: 5 LangGraph, 7 CrewAI, 7 AutoGen, and 8 ADK test cases (27 total). For each extractor, extracted graphs were compared against manually annotated ground truth, measuring node detection precision/recall, node-kind classification accuracy, and edge detection precision/recall.

Human-node detection limitation. The primary source of classification error is human-node detection in LangGraph, which relies on a naming heuristic ("**human**" in **name**): nodes not following this convention are misclassified as LLM nodes. Four strategies for human-node detection, in order of increasing reliability: (1) *Naming heuristic* (current default): match node names against keywords—low implementation cost but brittle. (2) *Interrupt annotation*: detect LangGraph’s **interrupt_before**/**interrupt_after** markers—framework-specific but semantically precise. (3) *Input-call detection*: identify **input()** or equivalent blocking calls in node functions via AST inspection—cross-framework but may produce false positives. (4) *Explicit type annotation*: require developers to mark human nodes via a decorator or metadata field—most reliable but requires adoption. On the test suite, only strategy (1) produces misclassifications; strategies (2)–(4) would require validation on a larger real-world corpus. Strategy (2) is prioritized for the next release as it requires no developer action.

Table 9: Static verification vs. runtime guardrails.

Property	Agentproof (static)	Runtime tools
Detection time	Pre-deployment	At execution
Structural defects	✓	Path-dependent
Toxic LLM output		✓
Runtime overhead	None	Per-call latency
Coverage	All paths	Exercised paths
Witness traces	✓	Stack traces

Extraction results. With the human-node caveat above, node detection achieves perfect precision and recall across all frameworks on the test suite. Node-kind classification accuracy is 100% for entry/exit sentinels and tool nodes (detected by tool bindings), and 100% for AutoGen agent types (detected by class hierarchy). Edge detection precision and recall are both 100% for all frameworks tested.

On the test suite, which uses programmatic framework stubs, extraction achieves perfect precision and recall for nodes and edges. This validates the extraction logic against known API patterns but does not measure robustness to the diversity of real-world workflow definitions. The AST-based extractor (`scripts/ast_extractor.py`) provides an independent extraction path; agreement between runtime and AST extraction on the same workflows would strengthen confidence and is a direction for future cross-validation.

8 Related work

This section situates the present work within four areas. Runtime tools catch content-level violations but miss structural defects. General-purpose model checkers could verify the same properties but require manual modeling. Temporal monitoring is well-studied but not applied to agent workflows. Agent safety research focuses on alignment, not orchestration topology. Agentproof fills the gap between runtime content checking and structural verification.

8.1 Runtime agent safety tools

Several tools enforce safety at the point of LLM output or tool invocation. NVIDIA NeMo Guardrails [23] interposes a programmable dialog rail between the LLM and tool calls, supporting topic control and output filtering. Guardrails AI [16] wraps LLM outputs with validators that check format, toxicity, and factual consistency. LlamaGuard [18] uses a fine-tuned classifier to detect unsafe content in model outputs. Rebuff [26] focuses specifically on prompt injection detection. These tools operate at runtime and catch violations only when the offending code path is actually exercised. Agentproof is complementary: it verifies structural properties of the workflow topology *before deployment*, catching defects such as unreachable exits, dead-end branches, and missing human gates that runtime tools cannot detect unless the defective path is triggered during testing.

Table 10: Comparison with general-purpose model checkers.

Tool	Input	Properties	Modeling	Time	Domain
SPIN	Promela (manual)	Full LTL	High	Fast (small)	None
NuSMV	SMV (manual)	CTL + LTL	High	Fast (small)	None
CBMC	C source	Assertions	Medium	Bounded	None
Agentproof	Auto-extracted	Safety LTL fragment	None	$O(V + E)$	Agent workflows

SPIN, NuSMV, and CBMC require manual translation of agent workflows into their input languages. Agentproof extracts models automatically from framework APIs, eliminating the modeling step entirely.

8.2 Static analysis and model checking

Classical model checking [11] verifies finite-state systems against temporal specifications using tools such as SPIN [17], NuSMV [10], and CBMC [20]. Abstract interpretation [12] provides sound over-approximations for dataflow analysis.

Table 10 provides a qualitative comparison. The key difference is *modeling effort*: SPIN requires manual translation of a workflow into Promela, NuSMV into SMV, and CBMC into annotated C. For the email triage workflow in the corpus (8 nodes, 7 edges), the equivalent Promela model requires ~60 lines and manual specification of the state space, transitions, and LTL properties (see `corpus/comparisons/email_triage.pml` in the artifact). Agentproof extracts the same model automatically with a single function call.

Agentproof applies model-checking ideas to a new domain: agent workflow graphs extracted from orchestration framework APIs. The contribution is not the graph algorithms themselves (which are standard) but (i) the automatic extraction from heterogeneous framework representations, (ii) a domain-specific property language tailored to agent safety, and (iii) empirical evidence that agent workflows contain structural defects catchable by these methods.

8.3 Temporal logic monitoring

The runtime verification (RV) community has developed sophisticated monitoring frameworks. JavaMOP [19] monitors Java programs against specifications in multiple formalisms. Bauer et al. [9] introduce three-valued LTL monitoring that distinguishes between “currently satisfied” and “permanently satisfied.” Barringer et al. [8] present the EAGLE framework for rule-based monitoring. The temporal monitors are deliberately lightweight, targeting the safety fragment of LTL with DFAs of $2-O(k)$ states per rule, trading expressiveness for deployment simplicity. In the evaluation, all 15 practical agent safety policies fall within this fragment, validating the design choice empirically (Section 7.3).

8.4 Agent architecture and safety

Recent work on AI agent safety spans multiple dimensions. Anthropic’s Responsible Scaling Policy [6] outlines safety commitments including evaluation thresholds for autonomous capabilities. METR [21] provides evaluations for autonomous agent capabilities and risks. Research on multi-agent coordination safety [15] examines the risks of agent-to-agent interaction, including unintended emergent behavior. These works focus on alignment, capability evaluation, and policy governance. The present contribution addresses a complementary layer: the *structural soundness of the orchestration graph itself*. Even a well-aligned LLM can produce unsafe outcomes if the workflow graph routes it

through an unintended path or bypasses a required approval step.

8.5 Business process verification

The business process management (BPM) community has extensively studied workflow soundness. Van der Aalst’s work on Petri-net-based soundness checking for workflow nets [29] ensures that every case can complete and no dead transitions exist properties closely analogous to Agentproof’s exit-reachability and dead-end checks. BPMN verification tools [13] apply similar analyses to industry-standard process models. This work differs in *domain*: agent workflow graphs have typed nodes (LLM, TOOL, HUMAN) with framework-specific semantics that BPM tools do not model, and the extraction pipeline targets programmatic API objects rather than visual process models.

8.6 Constrained decoding and structured generation

Constrained decoding enforces output structure at the token level: Guidance [22] and Outlines [28] compile grammars or JSON schemas into token masks that guarantee well-formed outputs. These techniques operate *within* a single LLM call, constraining what the model can generate. Agentproof operates at a different granularity: it constrains the *transitions between* computation steps (nodes), not the content generated within any single step. The two approaches are complementary.

9 Limitations and future work

Workflow structure vs. LLM semantics. The proposed approach verifies properties of the workflow *topology* and the event stream emitted by execution. It does not prove semantic properties of LLM outputs (e.g., factuality or intent), which depend on non-deterministic model behavior and prompt/context choices.

Extractor heuristics. Frameworks evolve quickly and expose different internal representations across versions. The extractors employ heuristics (e.g., recognizing human review nodes by naming conventions) that may not generalize to all codebases. The current validation uses 27 stub-based test cases (Section 7.7); the gap between stub-based validation and the diversity of real-world workflow definitions is not yet measured. The AST-based extractor (`scripts/ast_extractor.py`) provides an independent extraction path that could serve as a cross-validation tool to strengthen confidence in extraction fidelity.

Runtime graph mutation (T3). Some frameworks allow dynamic modification of the workflow graph during execution. If the post-mutation graph is not re-verified, static guarantees no longer hold. Integrating re-verification hooks into framework event systems is a natural extension.

Temporal DSL expressiveness. The DSL covers seven expression forms targeting the safety fragment of LTL but does not support full LTL, nested temporal operators beyond two levels, past-time modalities, or real-time bounds beyond step counting. All 15 evaluation policies fit within this fragment; however, these policies were authored by the same team that designed the DSL, introducing a self-selection bias: requirements that did not fit the grammar were unlikely to be proposed. The policies were inspired by external standards (GDPR [14], SOC 2 [5], OWASP LLM

Top 10 [24]), which provides partial mitigation, but a systematic survey of safety requirements from independent production deployments is needed to validate DSL sufficiency. Extending to richer temporal fragments (e.g., past-time LTL, real-time constraints) is future work.

False positive management. Some flagged defects may be intentional design choices (e.g., dead-end nodes used as intentional error-halting states, or workflows that intentionally omit human gates for low-risk tasks). Future work includes annotation-based suppression (e.g., `# agentproof: ignore dead-end` comments), severity tiers, and configurable verification profiles. The library already supports a `suppressions` parameter for node-level exclusions.

Static temporal conservatism. The graph $\times$ DFA product construction explores all topological paths, including paths that may be infeasible at runtime due to router conditions or LLM decision logic. This makes the analysis sound (no false negatives) but potentially conservative (false positives). Incorporating edge conditions into the product construction to prune infeasible paths is a direction for future work.

Corpus scale. The evaluation uses 18 author-constructed workflows. *Threat to validity:* because the corpus was designed to include representative defects and anti-patterns, the observed defect rates reflect detection capability on a targeted benchmark, not prevalence in production systems. Generalizing these rates requires validation on independently authored workflows. Agentproof provides a GitHub mining pipeline (`scripts/scrape_workflows.py`) and an AST-based fallback extractor (`scripts/ast_extractor.py`) as ready infrastructure for building larger real-world corpora. A large-scale study across hundreds of open-source repositories is needed to establish base rates and to provide a reusable benchmark for the community.

10 Conclusion

Agent frameworks already encode behavior as explicit workflow graphs. Agentproof leverages this structure to enable pre-deployment verification of safety properties without adding runtime overhead. Agentproof provides a unified graph model with six structural checks and witness traces, a temporal policy DSL covering the safety fragment of LTL with both static and runtime evaluation modes, and automatic extractors for four major agent frameworks—eliminating the manual modeling effort required by general-purpose model checkers. In the 18-workflow curated benchmark, 5 contain structural defects (dead ends, unreachable exits) and 10 lack a human gate when the policy is enforced. All 15 temporal safety policies evaluated fit within the seven-form DSL fragment, with verification completing in sub-second time even for graphs of 5,000 nodes.

The primary technical contribution is not the graph algorithms themselves—which are standard—but rather: (i) the identification of a practical new domain where these algorithms apply directly, (ii) the engineering of extractors that normalize four heterogeneous framework APIs into a single analyzable representation, and (iii) the empirical finding that real-world agent workflow patterns contain structural defects detectable by these methods.

Static verification does not replace runtime guardrails; the two approaches are complementary. By catching topology-level defects exhaustively before deployment, static analysis reduces the safety surface that runtime enforcement must cover, making both layers more effective.

Artifact availability. The Agentproof tool, curated corpus, temporal policies, and all evaluation scripts are available at <https://github.com/NordicAgents/AgentProof> under the MIT license. A reproducibility script (`scripts/reproduce_all.sh`) runs the complete evaluation pipeline.

References

- [1] AutoGen AgentChat. Python package `autogen-agentchat` (Microsoft), 2025. Accessed: 2026-02-28.
- [2] CrewAI documentation. <https://docs.crewai.com>, 2025. Accessed: 2026-02-28.
- [3] Google Agent Development Kit (ADK) documentation. <https://google.github.io/adk-docs/>, 2025. Accessed: 2026-02-28.
- [4] LangGraph documentation. <https://docs.langchain.com/oss/python/langgraph/overview>, 2025. Accessed: 2026-02-28.
- [5] AICPA. SOC 2 — trust services criteria. <https://www.aicpa.org/topic/audit-assurance/audit-and-assurance-greater-than-soc-2>, 2017. Accessed: 2026-03-01.
- [6] Anthropic. Anthropic’s responsible scaling policy. <https://www.anthropic.com/responsible-scaling-policy>, 2023. Accessed: 2026-03-01.
- [7] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. MIT Press, 2008.
- [8] Howard Barringer, Allen Goldberg, Klaus Havelund, and Koushik Sen. Rule-based runtime verification. In *International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI)*. Springer, 2004.
- [9] Andreas Bauer, Martin Leucker, and Christian Schallhart. Runtime verification for LTL and TLTL. *ACM Transactions on Software Engineering and Methodology*, 20(4), 2011.
- [10] Alessandro Cimatti, Edmund Clarke, Enrico Giunchiglia, et al. NuSMV 2: An opensource tool for symbolic model checking. In *International Conference on Computer Aided Verification (CAV)*. Springer, 2002.
- [11] Edmund M. Clarke, Orna Grumberg, and Doron A. Peled. *Model Checking*. MIT Press, 1999.
- [12] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs. In *4th ACM Symposium on Principles of Programming Languages (POPL)*, 1977.
- [13] Remco M. Dijkman, Marlon Dumas, and Chun Ouyang. Semantics and analysis of business process models in BPMN. *Information and Software Technology*, 50(12):1281–1294, 2008.
- [14] European Parliament and Council of the European Union. Regulation (EU) 2016/679 (GDPR), article 22: Automated individual decision-making, including profiling. <https://gdpr-info.eu/art-22-gdpr/>, 2016. Accessed: 2026-03-01.
- [15] Iason Gabriel et al. The ethics of advanced AI assistants. *arXiv preprint arXiv:2404.16244*, 2024.

- [16] Guardrails AI. Guardrails AI: Adding guardrails to large language models. <https://github.com/guardrails-ai/guardrails>, 2023. Accessed: 2026-03-01.
- [17] Gerard J. Holzmann. The model checker SPIN. *IEEE Transactions on Software Engineering*, 23(5):279–295, 1997.
- [18] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, et al. LlamaGuard: LLM-based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*, 2023.
- [19] Dongyun Jin, Patrick O’Neil Meredith, Choonghwan Lee, and Grigore Roşu. JavaMOP: Efficient parametric runtime monitoring framework. In *International Conference on Software Engineering (ICSE)*, 2012.
- [20] Daniel Kroening and Michael Tautschnig. CBMC – C bounded model checker. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer, 2014.
- [21] METR. METR: Model evaluation and threat research. <https://metr.org>, 2024. Accessed: 2026-03-01.
- [22] Microsoft. Guidance: A guidance language for controlling large language models. <https://github.com/guidance-ai/guidance>, 2023.
- [23] NVIDIA. NeMo Guardrails: A toolkit for controllable and safe LLM applications. <https://github.com/NVIDIA/NeMo-Guardrails>, 2023. Accessed: 2026-03-01.
- [24] OWASP Foundation. OWASP top 10 for large language model applications. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>, 2025. Version 2.0. Accessed: 2026-03-01.
- [25] Amir Pnueli. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science (FOCS)*, 1977.
- [26] Protectai. Rebuff: Self-hardening prompt injection detector. <https://github.com/protectai/rebuff>, 2023. Accessed: 2026-03-01.
- [27] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- [28] .txt. Outlines: Structured text generation. <https://github.com/outlines-dev/outlines>, 2023.
- [29] Wil M. P. van der Aalst. *Process Mining: Discovery, Conformance and Enhancement of Business Processes*. Springer, 2011.
- [30] Moshe Y. Vardi. Reasoning about the past with two-way automata. In *International Colloquium on Automata, Languages and Programming (ICALP)*, 1994.
- [31] Lei Wang, Chen Ma, Xueyang Feng, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024.

- [32] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Conference on Language Modeling (COLM)*, 2024.
- [33] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

A Soundness proofs

This appendix proves that each structural check is sound: if the check passes, the corresponding safety property holds for all valid execution traces.

Lemma 1 (Exit Reachability Soundness). *If $\text{ExitReach}(G)$ holds (i.e., the BFS/DFS from v_0 visits all of V_f), then for every exit node $v_f \in V_f$, there exists a directed path from v_0 to v_f in G .*

Proof. The check computes $\text{Reach}(v_0)$ by BFS from v_0 , which correctly identifies all nodes reachable via directed edges [11]. If $v_f \in \text{Reach}(v_0)$, then by the correctness of BFS there exists a path $v_0 \rightarrow \dots \rightarrow v_f$. Since the check passes only when $V_f \subseteq \text{Reach}(v_0)$, the result follows. $\square$

Lemma 2 (Dead-End Soundness). *If $\text{NoDead}(G)$ holds, then every non-exit node in every trace of G has at least one successor, and hence no maximal trace terminates at a non-exit node.*

Proof. Let v be any node with $\kappa_V(v) \neq \text{EXIT}$. Since $\text{NoDead}(G)$ holds, there exists u such that $(v, u) \in E$. Therefore, if a trace π visits v , the trace can be extended by u . A trace can only be maximal (i.e., cannot be extended) when it ends at a node in V_f or at a node with no outgoing edges. Since every non-exit node has outgoing edges, all maximal traces end at exit nodes. $\square$

Lemma 3 (Router Shape Soundness). *If $\text{RouterShape}(G)$ holds, then every outgoing transition from a router node is explicitly labeled as conditional.*

Proof. Direct from the predicate definition: the check iterates all edges $(v, u) \in E$ where $\kappa_V(v) = \text{ROUTER}$ and verifies $\kappa_E(v, u) = \text{CONDITIONAL}$. Passing the check establishes the universal quantification. $\square$

Lemma 4 (Human Gate Soundness). *If $\text{HumanGate}(G)$ holds, then the graph contains at least one node classified as a human-in-the-loop step.*

Proof. The check searches for $v \in V$ with $\kappa_V(v) = \text{HUMAN}$. Passing the check certifies existence. $\square$

Lemma 5 (Tool Declaration Soundness). *If $\text{ToolDecl}(G)$ holds, then every tool node explicitly declares its tool set.*

Proof. For each v with $\kappa_V(v) = \text{TOOL}$, the check verifies $T(v) \neq \emptyset$. Passing the check establishes the universal statement. $\square$

Lemma 6 (Reverse Reachability Soundness). *If $\text{ExitReachAll}(G)$ holds (i.e., $\text{Reach}(v_0) \subseteq \text{RevReach}(V_f)$), then for every node v reachable from v_0 , there exists a directed path from v to some exit node $v_f \in V_f$.*

Proof. The check computes $\text{RevReach}(V_f)$ by BFS on the reverse adjacency starting from all nodes in V_f . By the correctness of BFS, $v \in \text{RevReach}(V_f)$ implies the existence of a path $v \rightarrow \dots \rightarrow v_f$ for some $v_f \in V_f$. The check passes only when $\text{Reach}(v_0) \subseteq \text{RevReach}(V_f)$, so the result holds for every reachable node. $\square$

Lemma 7 (Human Gate Coverage Soundness). *If $\text{HumanGateCov}(G, S)$ holds for a set S of sensitive tool names, then every path from v_0 to a node invoking a tool in S passes through at least one HUMAN node.*

Proof. The check constructs a modified graph G' by removing all HUMAN-typed nodes and their incident edges, then computes $\text{Reach}_{G'}(v_0)$. If no sensitive tool node is in $\text{Reach}_{G'}(v_0)$, then in the original graph G every path from v_0 to such a node must traverse a removed HUMAN node. Conversely, if a sensitive tool node is reachable in G' , the check fails and reports the human-free path. $\square$

Lemma 8 (Static Temporal Soundness). *For a compiled monitor rule with DFA $(Q, q_0, \delta, F_{\text{viol}})$ and an agent workflow graph G , if the graph $\times$ DFA product construction reports no violation, then no execution trace of G violates the temporal property.*

Proof. The product BFS explores all reachable states in $V \times Q$. Each product state (v, q) represents being at graph node v with the DFA in state q . For each successor u of v in G , the DFA transitions via $q' = \delta(q, \sigma(v))$ where $\sigma(v)$ is the event symbol for node v , and the product successor (u, q') is enqueued. If no reachable product state has $q' \in F_{\text{viol}}$, then no execution trace — which corresponds to a path through product states — can reach a violation state. This is an over-approximation: the product explores all *graph paths*, which is a superset of actual runtime traces (since runtime traces depend on LLM decisions at router nodes). Hence the analysis is sound but potentially conservative. $\square$

Lemma 9 (Temporal Monitor Soundness). *For each of the three base DFA patterns (forbidden, implication-future, until), if the compiled monitor reports no violation on a finite trace π , then π satisfies the corresponding temporal formula.*

Proof. The proof proceeds by cases.

Forbidden ($\mathbf{G} \neg a$): The DFA has two states: s_0 (safe) and s_1 (violated, absorbing). The transition function maps $s_0 \xrightarrow{a} s_1$ and $s_0 \xrightarrow{\neg a} s_0$, with s_1 absorbing. If the monitor never enters s_1 , then a was false at every step, establishing $\mathbf{G} \neg a$.

Implication-future ($a \rightarrow \mathbf{F} b$): The DFA has three states: s_0 (idle), s_1 (waiting for b), and s_2 (violated, absorbing). Transition: $s_0 \xrightarrow{a \wedge \neg b} s_1$; $s_1 \xrightarrow{b} s_0$; $s_1 \xrightarrow{a \wedge \neg b} s_2$. If the monitor never enters s_2 , then every occurrence of a is followed by b before a recurs.

Until ($a \mathbf{U} b$): The DFA has three states: s_0 (waiting), s_1 (satisfied, absorbing), s_2 (violated, absorbing). Transition: $s_0 \xrightarrow{b} s_1$; $s_0 \xrightarrow{a \wedge \neg b} s_0$; $s_0 \xrightarrow{\neg a \wedge \neg b} s_2$. If the monitor never enters s_2 , then a held on every step until b occurred, establishing $a \mathbf{U} b$.

In each case the DFA state invariant is maintained by induction on the trace prefix. $\square$

B Temporal DSL grammar

The full BNF grammar for the temporal policy DSL:

```

<rule>          ::= <forbidden>
                  | <impl_future>
                  | <until>
                  | <bounded>
                  | <chain>
                  | <conjunction>
                  | <disjunction>

<forbidden>     ::= "G" "!" <atom>

<impl_future>   ::= <atom> "->" "F" <atom>

<until>         ::= <atom> "U" <atom>

<bounded>       ::= <atom> "->" "F" ["<=" <int> "]" <atom>

<chain>         ::= <atom> ("->" "F" <atom>){2,}

<conjunction>   ::= "(" <rule> ")" "AND" "(" <rule> ")"

<disjunction>   ::= "(" <rule> ")" "OR" "(" <rule> ")"

<atom>          ::= "tool:" <name>
                  | "action:" <name>
                  | "decision:" <name>
                  | <tag_name>

<name>          ::= [a-zA-Z_][a-zA-Z0-9_]*
<tag_name>      ::= [a-zA-Z_][a-zA-Z0-9_]*
<int>           ::= [1-9][0-9]*

```

Predicate matching. Atomic predicates are matched against event dictionaries: `tool:X` matches when the event's `tool_name` field equals `X`; `action:X` matches `action_type`; `decision:X` matches the `decision` field; bare names match membership in the event's `tags` set.